TuneTale: Genre-Aware Content-Based Music Recommendation Using K-Nearest Neighbors

Nathan Andrew So
naso7797@colorado.edu
University of Colorado at Boulder

Sharon Xiang
shxi3448@colorado.edu
University of Colorado at Boulder

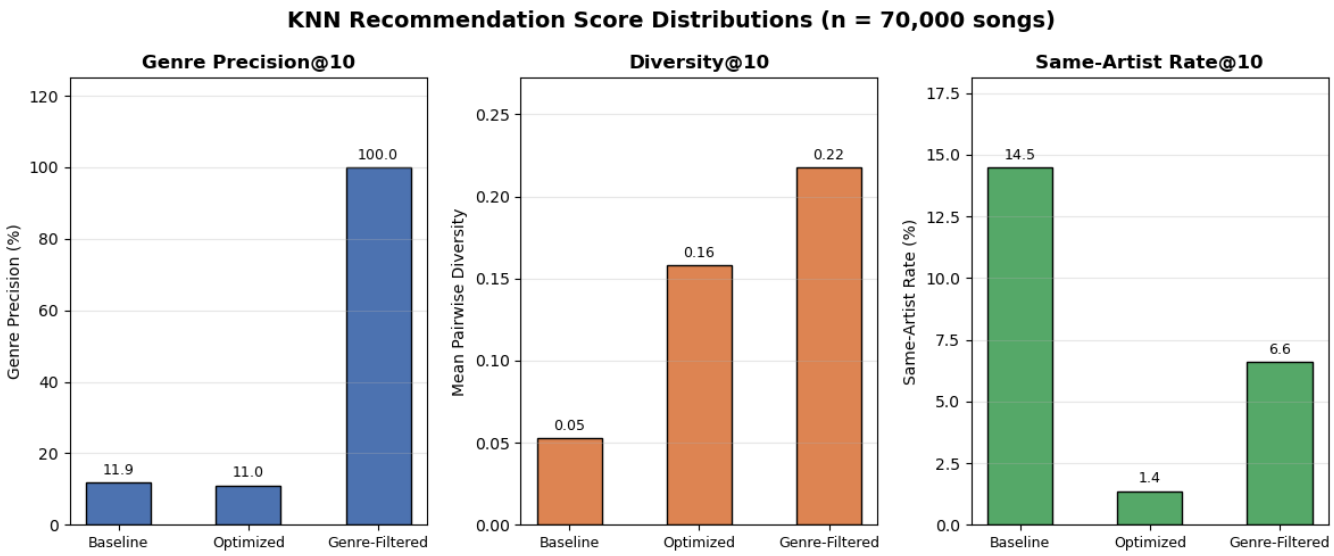

Figure 1: Genre precision, diversity, and same-artist rate across model stages (n = 70,000 queries, K = 10).

Abstract

Music discovery on Spotify is hard when the platform only knows what you've already listened to. TuneTale is a content-based recommender that uses K-Nearest Neighbors on audio features instead of listening history, so it works even with no user data. We trained on a 114,000-track Spotify dataset, reduced to 81,343 unique tracks after deduplication. Cleaning up duplicates turned out to be the biggest win: same-artist rate dropped from 14.5% to 1.4% and diversity tripled from 0.053 to 0.158, while genre precision stayed around 11%. We also added an optional genre filter that locks recommendations to the same genre as the input song, hitting 100% genre precision. The results show that data cleaning matters more than feature engineering here, and that genre filtering is the right tool when you want genre-consistent recommendations.

CCS Concepts

• **Information systems** → **Recommender systems**; • **Computing methodologies** → *Instance-based learning*; • **Applied computing** → Sound and music computing.

Code Repository: https://github.com/shxi3448/TuneTale

Kaggle Dataset: Spotify Tracks Dataset | Audio Features

1 Introduction

Spotify has over 100 million tracks, which makes finding something new genuinely hard. Most recommender systems use collaborative filtering, which looks at what similar users have listened to. That works well at scale, but breaks down for new users or niche genres where there isn't much history to work with. TuneTale takes a different approach: given a song, it finds 10 similar songs purely based on audio features like tempo, energy, and danceability, with no listening history required.

2 Related Work

Collaborative filtering recommends content by finding users with similar taste and surfacing what they liked. It powers recommendations at Netflix, Spotify, and most other large platforms. Content-based filtering is different: it recommends things similar to what a user has already liked, based on the items' own features rather than other users' behavior. Spotify actually uses a mix of both approaches, combining collaborative filtering with NLP on lyrics and audio analysis. TuneTale focuses specifically on the audio side of that pipeline.

3 Methodology

3.1 Dataset

We used a Kaggle dataset of 114,000 Spotify tracks across 114 genres, uniformly sampled at 1,000 tracks per genre. After removing duplicates by track ID and then by track name and artist, we were left with 81,343 unique tracks. We kept the pre-deduplication version (113,999 tracks) as a baseline to see how much the cleanup actually helped.

3.2 Feature Selection

We started with 5 features: danceability, energy, loudness, valence, and tempo. The results weren't great, so we expanded to 9 by adding acousticness, speechiness, instrumentalness, and liveness. These extra features have more variation across genres, which helps the model tell genres apart.

3.3 Normalization

All 9 features were normalized to $[0, 1]$ using MinMaxScaler. This is important because raw loudness ranges from -49.53 to 4.53 dB and tempo from 0 to 243.37 BPM. Without normalization, those features would completely dominate the distance calculation.

3.4 Algorithm

We used KNN ($K = 10$) with Euclidean distance and a ball tree index. Since all features are on the same scale after normalization, Euclidean and cosine distance are effectively equivalent here, and the ball tree makes the nearest-neighbor search efficient.

3.5 Genre-Aware Filtering

We added an optional mode that filters results to only include songs from the same genre as the query. It works by fetching a pool of $K \times 20 = 200$ neighbors and then keeping the top K that match the genre. This guarantees 100% genre precision, though it doesn't always have enough same-genre songs to work for every query.

4 Experiments and Evaluation

4.1 Evaluation Metrics

We measured three things: (1) **Genre Precision@K**, the fraction of the 10 recommendations that share the query song's genre; (2) **Diversity**, the mean pairwise Euclidean distance among the 10 recommendations, where higher means more varied; and (3) **Same-Artist Rate**, the fraction of recommendations by the same artist as the query, which we want to be low for good discovery. All metrics are averaged over $n = 70{,}000$ randomly sampled songs.

4.2 Baseline

The baseline used 5 features on the raw dataset with duplicates still in (113,999 tracks). It got 11.9% genre precision, diversity of 0.053, and a same-artist rate of 14.5%. That high same-artist rate was a red flag: a lot of the "similar" songs were just re-releases or live versions of the same track.

Table 1: Model performance across stages ($n = 70{,}000$ queries). Genre-filtered results are over the 59,704 queries (85.3%) with at least 2 same-genre neighbors.

Stage	Genre Precision	Diversity	Same Artist Rate
Baseline	11.9%	0.053	14.5%
Optimized	11.0%	0.158	1.4%
Genre-Filtered	100.0%	0.218	6.6%

4.3 Results

4.4 Discussion

Deduplication was by far the most impactful change. The duplicate entries (re-releases, remasters, live versions) were all by the same artist as the original, so they inflated the same-artist rate to 14.5%. After removing them, same-artist rate dropped to 1.4% and diversity tripled from 0.053 to 0.158. Genre precision barely moved (11.9% to 11.0%), so the 9-feature expansion didn't help much on that front, but the recommendation quality in terms of variety improved a lot.

Genre filtering pushed genre precision to 100% and diversity up to 0.218. The same-artist rate went back up to 6.6%, which makes sense. A smaller genre pool means fewer artists to draw from. About 14.7% of queries couldn't get genre-filtered results at all because their genre had too few tracks in the dataset.

4.5 Limitations

Genre labels are subjective. The same song could reasonably belong to multiple genres, and with 114 genres in the dataset, a lot of them are small niche categories where 11% genre precision is actually a high bar. The 9 audio features Spotify provides also can't capture everything: melody, timbre, and production style all matter for how a song sounds, and none of them are in the feature set. Genre filtering's 100% genre precision is also worth putting in context: it's guaranteed by how the filter works, not because the model learned anything. And without any human listening tests, we can't say how good these recommendations actually feel to a real user.

5 Ethical Considerations and AI Usage Disclosure

5.1 AI Usage Disclosure

Claude (Anthropic) was used as a coding assistant for debugging, fixing data preprocessing issues like deduplication, improving evaluation metrics, and helping with writing. All design decisions, analysis, and final content are the authors' own.

5.2 Dataset Bias

The dataset is evenly sampled at 1,000 tracks per genre across all 114 genres, so no genre starts out over-represented. The genre list covers a lot of ground beyond Western music: j-pop, k-pop, cantopop, mandopop, afrobeat, indian, iranian, malay, forro, samba, turkish, and more. That said, Spotify's genre labels are their own editorial choice, so some regional subgenres are probably missing or lumped in with something else.

After deduplication, the even sampling breaks down. Genres with a lot of re-releases lost the most tracks: reggaeton lost 93.7% of its 1,000 entries, and indie, house, rock, and jazz each lost over 78%. Niche regional genres like iranian (4.2% lost), malay (3.5%), and forro (3.2%) kept almost everything. This is about catalog re-release habits, not geography, but it does mean those heavily-duplicated genres end up with smaller same-genre pools and potentially weaker recommendations.